

DNS Architecture and Server Farm Routing

Juan David Guarnizo Hernandez, PhD
juan.guarnizo@tec.mx

TC2006B: Device Communications
Tecnológico de Monterrey
FJ2026

Goals

By the end of this comprehensive exercise you will:

1. Deploy a 3-Tier Enterprise Campus network featuring an isolated "Server Farm" architecture.
2. Configure a localized `dnsmasq` server to manage an internal enterprise DNS zone.
3. Route traffic between a user Access VLAN and a centralized Services VLAN.
4. Deploy enterprise application services (HTTP, FTP) and access them strictly using Fully Qualified Domain Names (FQDNs).

0) Pre-requirements & Software Installation

To simulate central DNS services, routing, and applications, ensure your system is updated and the necessary daemons and clients are installed:

```
1 sudo apt-get update
2 sudo apt-get install -y mininet dnsmasq tcpdump xterm curl ftp dnsutils python3-
  pyftplib
```

1) Designing the Internal DNS Zone

In an enterprise network, users navigate to applications using domain names (e.g., `web.corp.local`) rather than raw IP addresses. We will configure `dnsmasq` to act as an isolated, internal DNS resolver.

1.1 Creating the Isolated DNS Records

We must create a localized DNS zone file containing the FQDNs for our Server Farm. Using your preferred text editor (like `nano` or `vim`), create `/tmp/campus_dns_records.txt`:

```
1 192.168.100.10    dns.corp.local
2 192.168.100.20    web.corp.local
3 192.168.100.30    ftp.corp.local
```

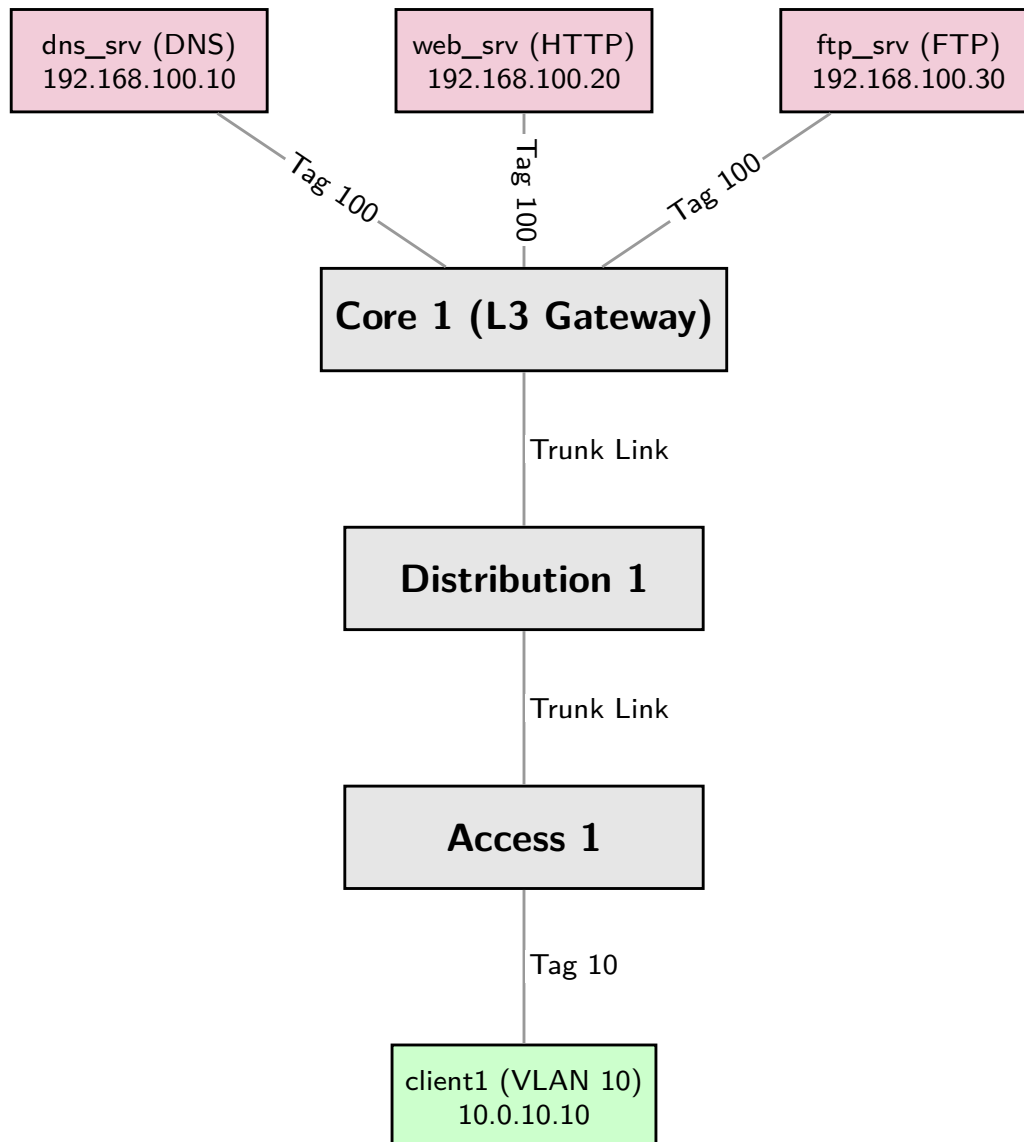


Figure 1: Target Architecture: A dedicated Server Farm (VLAN 100) integrated at the Core layer.

1.2 Appending the DNS Configuration

Next, append these DNS isolation parameters to the `/tmp/dhcp_campus.conf` file you created in the previous lab. By utilizing the `no-hosts` and `no-resolv` flags, we guarantee that this simulation will not conflict with your host Ubuntu machine's real internet connection.

```
1 no-hosts
2 no-resolv
3 log-queries
4 addn-hosts=/tmp/campus_dns_records.txt
```

2) Building the Campus Topology

We will create a Python script that provisions our Server Farm directly onto the Core switch. To ensure that our client host actually queries our new DNS server (instead of asking Google), we will use Mininet's `privateDirs` feature to safely overwrite the client's `/etc/resolv.conf` file.

Create a file named `campus_dns.py` and add the following code:

```
1 #!/usr/bin/env python3
2 from mininet.net import Mininet
3 from mininet.node import OVSSwitch
4 from mininet.link import TCLink
5 from mininet.cli import CLI
6 from mininet.log import setLogLevel
7
8 def build_campus():
9     net = Mininet(controller=None, switch=OVSSwitch, link=TCLink, autoSetMacs=
10         True, autoStaticArp=False)
11
12     core_1 = net.addSwitch('core1')
13     dist_1 = net.addSwitch('dist1')
14     acc_1 = net.addSwitch('acc1')
15
16     dns_srv = net.addHost('dns_srv', ip='192.168.100.10/24', defaultRoute='via
17         192.168.100.254')
18     web_srv = net.addHost('web_srv', ip='192.168.100.20/24', defaultRoute='via
19         192.168.100.254')
20     ftp_srv = net.addHost('ftp_srv', ip='192.168.100.30/24', defaultRoute='via
21         192.168.100.254')
22
23     client = net.addHost('client1', ip=None, privateDirs=['/etc'])
24
25     net.addLink(dns_srv, core_1, port1=0, port2=1)
26     net.addLink(web_srv, core_1, port1=0, port2=2)
27     net.addLink(ftp_srv, core_1, port1=0, port2=3)
28
29     net.addLink(client, acc_1, port1=0, port2=1)
30     net.addLink(acc_1, dist_1)
31     net.addLink(dist_1, core_1)
32
33     net.start()
34
35     acc_1.cmd('ovs-vsctl set port acc1-eth1 tag=10')
36     core_1.cmd('ovs-vsctl set port core1-eth1 tag=100')
37     core_1.cmd('ovs-vsctl set port core1-eth2 tag=100')
38     core_1.cmd('ovs-vsctl set port core1-eth3 tag=100')
39
40     core_1.cmd('ovs-vsctl add-port core1 vlan10 tag=10 -- set interface vlan10
41         type=internal')
42     core_1.cmd('ip addr add 10.0.10.254/24 dev vlan10')
43     core_1.cmd('ip link set vlan10 up')
44
45     core_1.cmd('ovs-vsctl add-port core1 vlan100 tag=100 -- set interface
46         vlan100 type=internal')
47     core_1.cmd('ip addr add 192.168.100.254/24 dev vlan100')
48     core_1.cmd('ip link set vlan100 up')
49
50     core_1.cmd('sysctl -w net.ipv4.ip_forward=1')
51     core_1.cmd('dhcrelay -4 -i vlan10 -i vlan100 192.168.100.10')
52
53     CLI(net)
54
55     core_1.cmd('killall dhcrelay 2>/dev/null')
56     net.stop()
```

```
52 if __name__ == '__main__':
53     setLogLevel('info')
54     build_campus()
```

3) Initializing Servers and Validating FQDN Access

Run your script: `sudo python3 campus_dns.py`.

We will now start the application daemons in the Server Farm and test them from the client across the routing boundary.

3.1 Start the Application Servers

Open three xterms for the servers from the Mininet CLI:

```
1 mininet> xterm dns_srv web_srv ftp_srv
```

In the `dns_srv` terminal, launch `dnsmasq` in the foreground (`-d`) using our custom configuration file. Leave this terminal open so you can watch the incoming queries!

```
1 # dnsmasq -d --conf-file=/tmp/dhcp_campus.conf --pid-file=/tmp/dhcp_campus.pid
```

In the `web_srv` terminal, launch the Python HTTP server:

```
1 # mkdir -p /tmp/web && echo "<h1>Welcome to web.corp.local</h1>" > /tmp/web/
  index.html
2 # cd /tmp/web && python3 -m http.server 80
```

In the `ftp_srv` terminal, launch the Python FTP server:

```
1 # mkdir -p /tmp/ftp && cd /tmp/ftp
2 # python3 -m pyftplib -p 21 -w -u admin -P secret123
```

3.2 DHCP Lease and FQDN Service Testing

Now that our Server Farm is online, we must request an IP address and DNS settings for our client. Return to the main Mininet CLI and execute the DHCP client:

```
1 mininet> client1 dhclient -v client1-eth0
```

Verify that the client successfully received an IP in the `10.0.10.x` range, and check that the DNS server was automatically configured inside its isolated namespace:

```
1 mininet> client1 cat /etc/resolv.conf
```

Next, open an xterm for your client:

```
1 mininet> xterm client1
```

Inside the `client1` terminal, first verify that standard IP connectivity works across the Core router:

```
1 # ping -c 2 192.168.100.20
```

Next, use `dig` to verify that the domain names are resolving correctly. We will first run an explicit query directed specifically at our internal DNS server (`@192.168.100.10`). This is a vital troubleshooting step: it ensures the server itself is functioning, bypassing the local `resolv.conf` entirely just in case the automatic DHCP injection failed:

```
1 # dig @192.168.100.10 web.corp.local
2 # dig @192.168.100.10 ftp.corp.local
```

Look for the *ANSWER SECTION* to confirm the IPs 192.168.100.20 and 192.168.100.30 are returned. Also observe the live query logs in the *dns_srv* xterm window!

Finally, test standard resolution relying natively on the *resolv.conf* file, and consume the services using strictly their Fully Qualified Domain Names (FQDNs):

```
1 # dig web.corp.local
2 # curl -s http://web.corp.local
3 # ftp ftp.corp.local
```

4) Reflection Questions

1. **Architecture Topography:** In this lab, the servers were placed directly off the Core switch rather than on an Access switch alongside the clients. From an enterprise performance and routing perspective, why is the "Server Farm" typically attached directly to the high-speed Core?
2. **Inter-VLAN Routing:** When *client1* (10.0.10.10) queried *dns_srv* (192.168.100.10), the packet successfully crossed the Layer 3 boundary. Which specific device and interface served as the default gateway for the DNS server's return traffic?
3. **DNS Architecture:** We explicitly configured *dnsmasq* with the *no-hosts* and *no-resolv* flags in Section 1.2. Why is it critical to isolate an enterprise DNS simulation from the host Linux operating system's DNS settings?
4. **Dynamic Configuration:** We completely removed static DNS configurations from the client. When *client1* ran *dhclient*, how did it automatically know to use 192.168.100.10 as its DNS server instead of a public server like 8.8.8.8? What specific protocol mechanism facilitated this?